

Forma **TEC**

CLASE 3

Contenedores (Docker) **Parte 2**



Previamente

Recordemos

Comentarios o dudas sobre la última clase:

- Introducción containers
- Docker

Repasamos Flujo completo docker

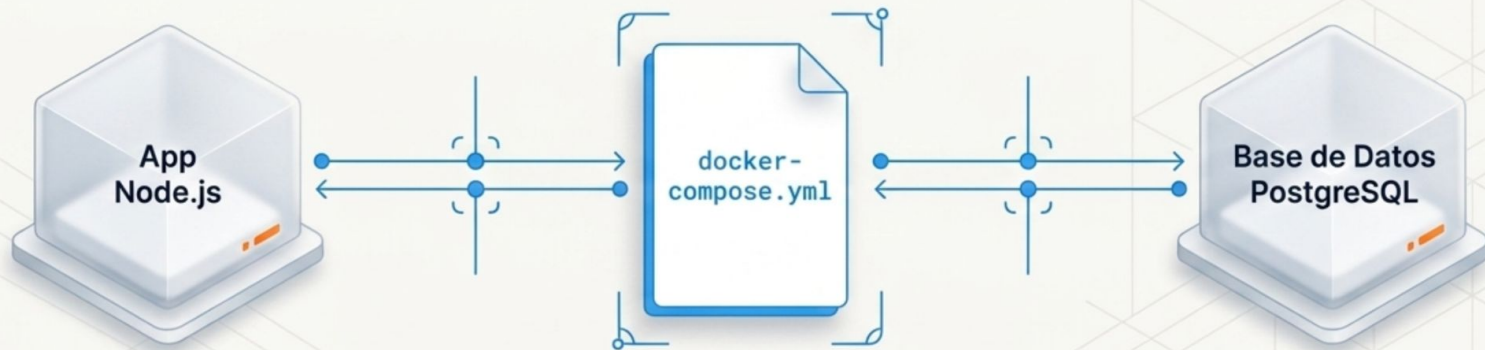


Fundamentos

Docker-compose

Las aplicaciones modernas rara vez son un solo contenedor. Imagina una aplicación Node.js que necesita interactuar con una base de datos PostgreSQL. En lugar de configurar y conectar cada contenedor a mano, utilizamos la orquestación.

El archivo `docker-compose.yml` vive en la raíz del proyecto y actúa como el director de logística. Con un solo comando, mueve el mundo y levanta todos los servicios interconectados simultáneamente.



Evolución Networking

Bridge (Por defecto)

Crea una interfaz de red virtual aislada en tu computadora. Los contenedores se comunican internamente usando su nombre como DNS sin exponer puertos al exterior.

Host

Elimina el aislamiento de red. El contenedor interactúa directo con la interfaz física y los puertos reales de la máquina anfitriona sin traducción de red.

None

Aislamiento total. El contenedor carece de conexión a redes y puertos externos, bloqueando cualquier intercambio de información externa.

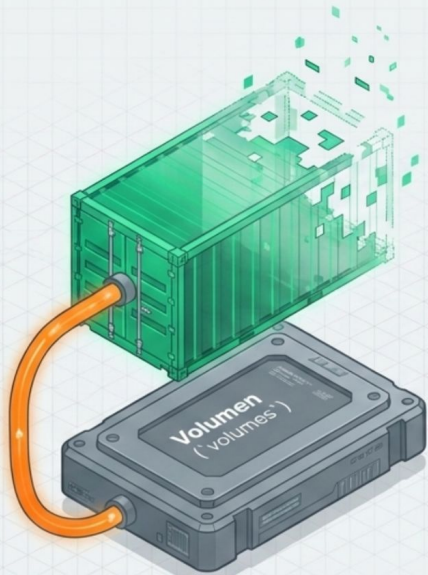
Overlay

Red distribuida. Permite conectar contenedores corriendo en diferentes computadoras físicas independientes (La base de Kubernetes y Docker Swarm).

Evolución

Volúmenes

Volúmenes



**"Los contenedores son efímeros,
los datos deben ser permanentes".**

El Problema:

Por defecto, si borras o actualizas un contenedor, todos los datos generados en su interior desaparecen.

La Solución:

Los Volúmenes de Docker. Son sistemas de archivos montados desde la máquina anfitriona hacia el contenedor, aislando el almacenamiento del ciclo de vida del contenedor (Vital para Bases de Datos).

Buenas prácticas

Seguridad

Optimización de Tamaño



Regla: Usa imágenes base mínimas y Multi-stage builds.

¿Por qué? Imágenes pequeñas se descargan más rápido, ocupan menos espacio y reducen la superficie de ataque.

Privilegios Restringidos



Regla: Nunca ejecutes aplicaciones como usuario root.

¿Por qué? Como el contenedor comparte el Kernel del Host, un contenedor comprometido con privilegios root amenaza toda la máquina servidor.

Escaneo Continuo



Regla: Escanea imágenes durante el desarrollo y en producción.

¿Por qué? Las vulnerabilidades de las bibliotecas cambian a diario. Mantén un registro (SBOM) actualizado.

Buenas prácticas

Optimización imágenes

```
# ETAPA 1: Construcción (Imagen pesada)
FROM node:18 AS builder
WORKDIR /app
COPY . .
RUN npm run build # Compila la app

# ETAPA 2: Producción (Imagen ultra-ligera)
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Dividir y Vencerás

Para compilar código de Angular, React o TypeScript necesitas NPM, SDKs y compiladores pesados. Sin embargo, para ejecutar el resultado solo necesitas los archivos de salida estáticos (`dist`).

Multi-stage: Te permite compilar en una imagen pesada intermedia y copiar únicamente el resultado final optimizado a una imagen limpia de producción (Nginx Alpine), reduciendo el peso final de ****800 MB a solo 30 MB****.

Cierre

Preguntas o Dudas

Próximos pasos

Manos a la obra

Herramientas o apps

- Seguimos con Docker